

# Teorie minimală — abstract

Pornind de la codul tău: `Figura` și `Interval` · Java · Adrian

Patru pagini de teorie n-ai nevoie aici. `abstract` face **două** lucruri, iar restul sunt consecințe ale acestor două.

## 1. De unde vine nevoia — două locuri din codul tău

### Metode scrise doar ca să existe

```
public class Figura {  
    public void afisare() { }  
    public void translate(int x, int y) { }  
    public Figura duplicare() { return null; }  
}
```

Corpurile astea nu fac nimic. Le-ai scris ca subclasele să aibă ce suprascrie și ca `Desen` să poată ține `List<Figura>`. Corect ca intenție — dar are două scurgeri:

- **Poți crea obiectul:** `new Figura()` compilează. Îl pui într-un desen, îl afișezi și primești un rând fără nicio informație. Un desen care conține „o figură în general” n-are sens.
- **Poți uita o metodă:** o figură nouă în care uiți `duplicare()` moștenește `return null`. Compilează, rulează, și îți dă `null` la prima duplicare.

Același lucru în orar: `Interval.getTipInterval()` întoarce `""`. Un interval al cărui tip nu e scris nicăieri se afișează cu un rând gol în dreptul tipului, fără ca nimeni să se plângă.

## 2. Ce face `abstract` — exact două lucruri

### Pe clasă: nu se mai pot crea obiecte din ea

```
public abstract class Figura { ... }  
  
Figura f1 = new Figura(); // eroare de compilare  
Figura f2 = new Cerc(...); // corect
```

Tipul `Figura` rămâne folosibil peste tot — ca variabilă, ca element în `List<Figura>`, ca parametru, ca tip returnat. Singurul lucru interzis e **fabricarea** unui obiect din el.

### Pe metodă: o declari fără corp, iar subclasele sunt obligate s-o scrie

```
public abstract class Figura {  
    public abstract void afisare();  
    public abstract void translate(int x, int y);  
    public abstract Figura duplicare();  
}
```

Nu mai există `{ }` și nu mai există `return null`. De acum, o subclasă care nu le scrie **nu compilează**. Greșeala se mută de la rulare — unde o descoperi târziu, dintr-un rezultat ciudat — la compilare, unde o vezi imediat și pe nume.

### 3. Ce are voie o clasă abstractă

Contrar a ce sună numele, o clasă abstractă nu e o listă de metode goale. E o clasă **normală, neterminată**:

| Are voie                                        | Exemplu în ierarhia ta                                                    |
|-------------------------------------------------|---------------------------------------------------------------------------|
| câmpuri, inclusiv <code>private</code>          | <code>oraInceput</code> , <code>durataMin</code> în <code>Interval</code> |
| constructor (des făcut <code>protected</code> ) | <code>Interval(int oraInceput, int durataMin)</code>                      |
| metode scrise complet                           | <code>decalare(int nrMinute)</code>                                       |
| metode doar declarate ( <code>abstract</code> ) | <code>abstract String getTipInterval();</code>                            |
| <b>NU are voie</b> : să fie instanțiată         | <code>new Interval(8, 50)</code>                                          |

Constructorul ei **chiar rulează** — de fiecare dată când creezi o subclasă, prin `super(...)`. Nu-l cheamă tu direct niciodată, dar fără el partea moștenită n-ar fi inițializată.

#### Piesa care leagă tot: o metodă scrisă poate chema una nescrisă

```
public abstract class Interval {  
    public abstract String getTipInterval();  
  
    public String afisare() {  
        return getTipInterval() + ": ora " + ...;  
    }  
}
```

Formatul rândului e scris **o singură dată**, în clasa de sus, deși `getTipInterval()` nu e scrisă nicăieri acolo. La rulare se cheamă versiunea obiectului real — exact polimorfismul pe care îl știi. Asta e adevărata întrebuințare a clasei abstracte: **ține algoritmul comun și lasă găurile în seama subclaselor**.

### 4. Regula lanțului și trei combinații interzise

#### Cine e obligat să fie abstract

- O clasă cu **măcar o** metodă abstractă trebuie declarată `abstract`.
- O subclasă care **nu le scrie pe toate** cele moștenite rămâne și ea abstractă — și trebuie declarată ca atare. Abia clasa care le termină pe toate poate fi normală.

| Scris                                   | De ce nu merge                                                       |
|-----------------------------------------|----------------------------------------------------------------------|
| <code>final abstract class X</code>     | <code>abstract</code> cere subclase, <code>final</code> le interzice |
| <code>private abstract void f();</code> | subclasa n-o vede, deci n-are cum s-o scrie                          |
| <code>abstract void f() { }</code>      | ori o declari, ori o scrii — nu amândouă                             |

**Cum decizi, în practică:** dacă poți scrie un corp care are sens pentru **toate** subclasele, scrie-l în clasa de sus. Dacă nu poți — dacă ai ajuns la `{ }`, la `return null` sau la `return ""` doar ca să treacă de compilator — aia e o metodă `abstract`. Corpul gol e semnul.